Client / contexte du projet 1
Objectif principal 1
Parties prenantes 1
Les principales parties prenantes de MangaTrack 1
Besoins exprimés 2
Gestion du compte 2
Catalogue mangas et animes 2
Bibliothèque personnelle 2
Suivi de progression 2
Suivi des sorties 3
Rappels configurables 3
Plan de rattrapage 3
Recommandations 3
Lien manga ↔ anime 4
Où lire ou regarder ? 4
Statistiques personnelles 4
Hypothèses et contraintes 5
Scénarios de qualité et arbre d'utilité 6
Scénario 1 — Performance 7
Scénario 2 — Sécurité 7
Scénario 3 — Modifiabilité 8
Scénario 4 — Disponibilité des rappels 8
Scénario 5 — Disponibilité du catalogue externe 9
Scénario 6 — Interopérabilité 9
Arbre d'utilité MangaTrack 10
Pattern principal et tactiques architecturales 10
Choix du pattern principal 10
Compromis accepté 12
Alternative étudiée : microservices 12
Documentation de l'architecture 12
C4 niveau 1 — Contexte 12
C4 niveau 2 — Conteneurs 13
C4 niveau 3 — Composants de l'API MangaTrack 13
Vue de déploiement cloud et edge 14
Architecture Decision Records 15
ADR-001 — Monolithe modulaire 15
ADR-002 — Ports et adaptateurs pour les API externes 15
ADR-003 — Notifications asynchrones 16
ADR-004 — Cache Redis pour le catalogue 16
ADR-005 — Fonctionnement hors ligne limité 16
Analyse économique 17
Risques restants, dette et évolution 18
Conclusion 19

Client / contexte du projet

Une jeune entreprise souhaite développer MangaTrack, une plateforme Web destinée aux lecteurs de mangas et aux spectateurs d'animes.
Aujourd'hui, les utilisateurs doivent souvent utiliser plusieurs services différents pour suivre leur progression, connaître les prochaines sorties, recevoir des rappels ou trouver une plateforme légale de lecture ou de visionnage.

MangaTrack cherche à centraliser ces usages dans une seule application personnalisée.

Objectif principal

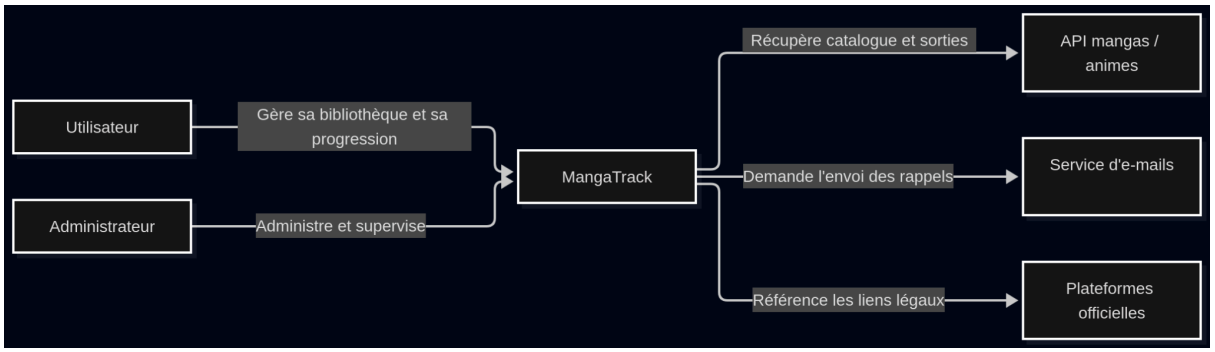
Permettre à un utilisateur de suivre facilement les mangas et animes qu'il consulte, sa progression et leurs prochaines sorties, tout en lui proposant des rappels et des fonctions personnalisées.

Parties prenantes

Les principales parties prenantes de MangaTrack

- **Utilisateur** : utilise l'application pour gérer sa bibliothèque, suivre sa progression, consulter les prochaines sorties et recevoir des rappels.
- **Administrateur** : supervise la plateforme, peut corriger certaines informations et gérer les problèmes liés aux services externes.
- **Fournisseur de données mangas/animes** : fournit les informations sur les œuvres, les épisodes, les chapitres et certaines dates de diffusion.
- **Service d'e-mails** : permet l'envoi des rappels et des récapitulatifs.
- **Plateformes officielles de lecture ou de visionnage** : permettent de rediriger l'utilisateur vers des contenus accessibles légalement.
- **Équipe de développement / exploitation** : maintient MangaTrack, surveille son fonctionnement et fait évoluer l'architecture.

Partie prenante	Attente principale	Impact architectural
Utilisateur	Application rapide et simple	Performance, disponibilité
Administrateur	Pouvoir superviser et corriger	Gestion des droits, observabilité
API externe	Échanges fiables	Ports/adaptateurs, cache, retry
Service e-mail	Gestion fiable des rappels	File de messages, worker
Plateformes officielles	Redirection correcte	Interopérabilité
Équipe technique	Application facile à faire évoluer	Monolithe modulaire, tests



Graphique parties prenantes

Besoins exprimés

Gestion du compte

- Créer un compte et se connecter,
- Gérer son profil,
- Gérer ses préférences,
- Choisir la confidentialité de certaines informations,
- Configurer ses notifications.

Catalogue mangas et animes

- Rechercher une œuvre,
- Consulter sa fiche,
- Afficher son titre, son genre, ses thèmes, son auteur ou studio,
- Afficher le nombre d'épisodes, chapitres ou tomes lorsqu'il est connu,
- Récupérer une partie des informations depuis une API externe.

Bibliothèque personnelle

L'utilisateur peut ajouter une œuvre avec différents statuts :

- A commencer,
- En cours,
- Terminé,
- Abandonné,
- Favori.

Suivi de progression

Pour un anime :

> Episode 7 sur 12.

Pour un manga :

> Chapitre 85 ou tome 9.

L'utilisateur peut également :

- Indiquer une date de début,
- Indiquer une date de fin,
- Donner une note,
- Rédiger un avis personnel.

Suivi des sorties

MangaTrack permet de connaître :

- La prochaine diffusion d'un épisode,
- Le numéro du prochain épisode,
- Les prochaines sorties manga lorsque l'information est disponible,
- Les sorties de la semaine,
- Un calendrier personnalisé basé sur les œuvres suivies.

Rappels configurables

L'utilisateur peut suivre une œuvre et choisir ses rappels :

- Une semaine avant,
- 24 h avant,
- 1 h avant,
- Le jour de la sortie,
- Au moment de la disponibilité,
- Résumé hebdomadaire.

Les rappels peuvent être envoyés par e-mail.

Plan de rattrapage

MangaTrack utilise la progression de l'utilisateur et la date de la prochaine sortie pour proposer un rythme.

Exemple :

> Tu as 5 épisodes de retard.
> Le prochain épisode sort dans 10 jours.
> Regarde environ un épisode tous les 2 jours pour être à jour.

L'utilisateur peut ensuite recevoir des rappels associés à ce programme.

Recommandations

Sur la fiche d'une œuvre, MangaTrack propose des œuvres similaires selon :

- Genres,
- Thèmes,
- Auteur,
- Studio,

- Favoris,
- Notes données,
- Historique de l'utilisateur.

Pour le MVP, on pourra limiter les recommandations aux **genres et tags similaires**, plutôt que développer un moteur complexe.

Lien manga ↔ anime

MangaTrack peut indiquer :

- De quel manga provient un anime,
- Quelle adaptation anime correspond à un manga,
- Les saisons correspondantes,
- Les chapitres ou tomes adaptés lorsqu'ils sont connus,
- A quel chapitre ou tome poursuivre après avoir terminé l'anime.

Où lire ou regarder ?

MangaTrack ne diffuse pas directement les œuvres.

Il fournit des liens vers des **plateformes officielles** et indique, lorsque l'information est disponible :

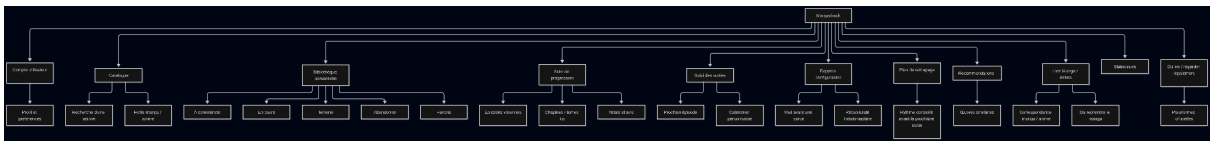
- Gratuit,
- Abonnement,
- Achat ou location.

Ça évite aussi de transformer MangaTrack en plateforme de streaming ou d'hébergement de mangas.

Statistiques personnelles

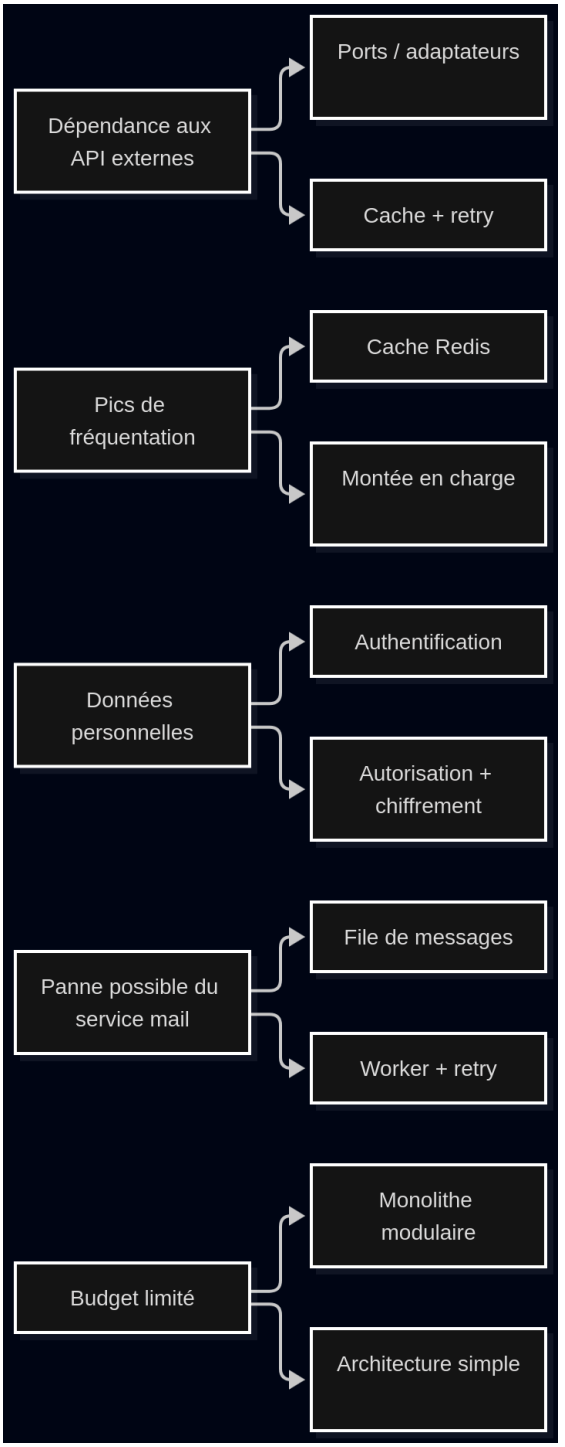
L'utilisateur peut consulter :

- Nombre d'animes terminés,
- Nombre de mangas lus,
- Episodes visionnés,
- Chapitres ou tomes lus,
- Temps estimé de visionnage,
- Genres les plus suivis,
- Moyenne des notes.



Carte fonctionnelle MangaTrack

Hypothèses et contraintes



Contraintes / Impact architecturaux

Dépendance à des services externes

MangaTrack ne possède pas l'ensemble du catalogue.

Il dépend d'une ou plusieurs API externes pour récupérer :

- Titres,
- Images,
- Genres,
- Informations sur les œuvres,
- Dates de diffusion.

Conséquence architecturale : il faut isoler ces API derrière des ports/adaptateurs pour pouvoir changer de fournisseur.

Les API peuvent être indisponibles

Une API externe peut ralentir ou tomber temporairement en panne.

Hypothèse :

> Une indisponibilité temporaire de l'API de catalogue ne doit pas rendre toute l'application inutilisable.

On pourra donc prévoir :

- Cache,
- Timeout,
- Retry,
- Eventuellement circuit breaker.

Pics de consultation

Une sortie très attendue peut provoquer beaucoup de consultations en peu de temps.

Hypothèse actuelle :

> MangaTrack doit supporter jusqu'à **5 000 utilisateurs simultanés** sur une œuvre populaire.

Données personnelles

MangaTrack stocke notamment :

- Adresse e-mail,
- Bibliothèque personnelle,
- Progression,
- Préférences,
- Notes,
- Paramètres de notifications.

Ces données devront être protégées et les accès contrôlés.

Les rappels ne doivent pas dépendre directement du fournisseur mail

Une panne du fournisseur d'e-mails ne doit pas faire disparaître les rappels.

D'où notre choix :

API MangaTrack → file de messages → worker → fournisseur e-mail

Budget limité

On reprend une contrainte proche du projet LES :

> Le projet dispose d'un budget d'hébergement limité.

Ça justifie notamment :

- Monolithe modulaire plutôt que microservices,
- Services managés raisonnables,
- Mise en cache ciblée,
- Architecture simple.

Contenus protégés

MangaTrack ne stocke ni épisodes ni chapitres protégés.

La plateforme fournit uniquement :

- Informations sur les œuvres,
- Progression,
- Recommandations,
- Liens vers des plateformes officielles.

Scénarios de qualité et arbre d'utilité

Les besoins fonctionnels indiquent ce que MangaTrack doit permettre de faire. Les scénarios de qualité permettent de préciser comment le système doit se comporter dans

différentes situations. Chaque scénario est défini par une source, un stimulus, un artefact concerné, un environnement, une réponse attendue et une mesure vérifiable.

Scénario 1 — Performance

Élément	MangaTrack
Source	Utilisateurs
Stimulus	5 000 utilisateurs consultent simultanément la fiche d'une œuvre populaire
Artefact	Catalogue et API MangaTrack
Environnement	Pic de fréquentation lors d'une sortie importante
Réponse	MangaTrack continue d'afficher les fiches et les informations de sortie
Mesure	95 % des pages s'affichent en moins de 2 secondes

Priorité : (H,H)

Scénario 2 — Sécurité

Élément	MangaTrack
Source	Utilisateur authentifié
Stimulus	Il tente d'accéder à la bibliothèque privée d'un autre utilisateur
Artefact	Comptes et données personnelles
Environnement	Fonctionnement normal
Réponse	MangaTrack refuse l'accès et journalise la tentative
Mesure	0 donnée privée transmise à l'utilisateur non autorisé

Priorité : (H,M)

Scénario 3 — Modifiabilité

Élément	MangaTrack
Source	Équipe de développement
Stimulus	L'API de catalogue utilisée doit être remplacée
Artefact	Module d'intégration avec les services externes
Environnement	Maintenance normale
Réponse	Le nouvel adaptateur est ajouté sans modifier les fonctions principales
Mesure	Nouvelle API intégrée en moins de 3 jours et modifications limitées au module d'intégration

Priorité : (M,M)

Scénario 4 — Disponibilité des rappels

Élément	MangaTrack
Source	Fournisseur d'e-mails
Stimulus	Le service d'e-mails devient indisponible pendant 30 minutes
Artefact	Système de notifications
Environnement	Des rappels sont programmés pendant la panne
Réponse	Les rappels restent dans la file et sont envoyés après le retour du service
Mesure	0 rappel perdu et 99 % des rappels envoyés dans les 5 minutes après le retour du service

Priorité : (H,H)

Scénario 5 — Disponibilité du catalogue externe

Élément	MangaTrack
Source	API externe mangas/animes
Stimulus	L'API externe ne répond plus pendant 20 minutes
Artefact	Catalogue MangaTrack
Environnement	Fonctionnement normal
Réponse	MangaTrack affiche les données déjà disponibles en cache et indique que certaines informations peuvent être temporairement anciennes
Mesure	Les œuvres déjà mises en cache restent consultables pendant toute la panne

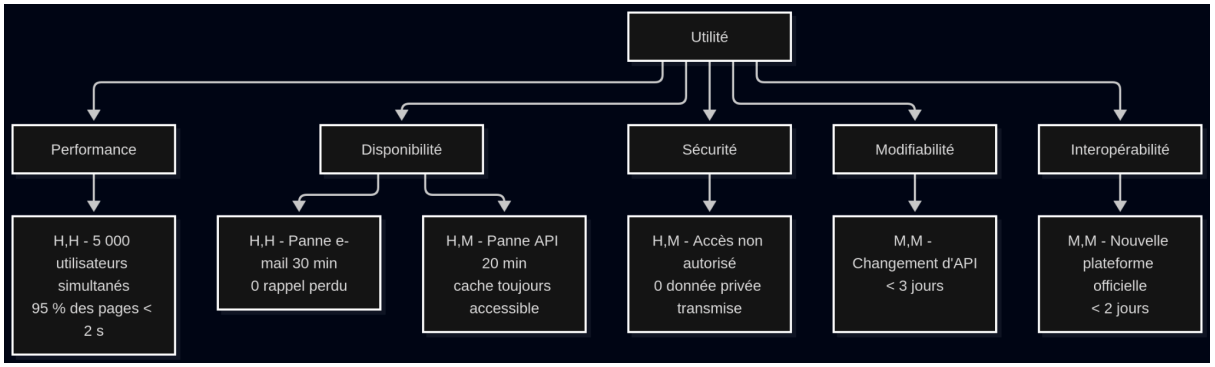
Priorité : (H,M)

Scénario 6 — Interopérabilité

Élément	MangaTrack
Source	Équipe de développement
Stimulus	Une nouvelle plateforme officielle de lecture ou de visionnage doit être ajoutée
Artefact	Module « Où lire ou regarder ? »
Environnement	Évolution normale du produit
Réponse	La nouvelle plateforme est intégrée sans modifier la bibliothèque ou le suivi de progression
Mesure	Intégration réalisée en moins de 2 jours et limitée au module des plateformes externes

Priorité : (M,M)

Arbre d’utilité MangaTrack



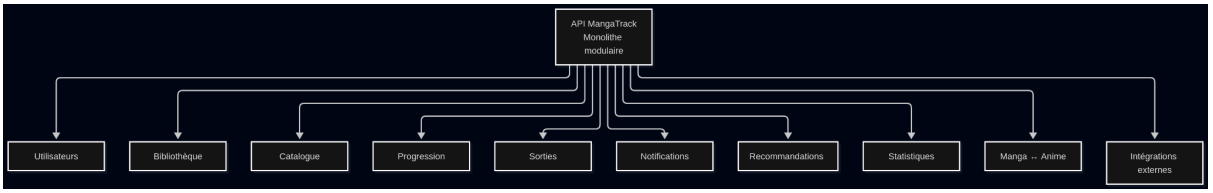
Pattern principal et tactiques architecturales

Choix du pattern principal

MangaTrack adopte comme pattern principal une architecture en **monolithe modulaire**. L’application est déployée comme une seule unité, mais son code est séparé en modules correspondant aux grandes fonctions du système.

Ce choix permet de garder une architecture relativement simple à développer et exploiter tout en limitant les dépendances entre les différentes fonctions. Il correspond également au contexte d’une petite équipe et à la contrainte de budget définie pour le projet.

Les frontières entre les modules doivent rester explicites afin qu’un module puisse évoluer sans entraîner de modifications inutiles dans les autres parties de MangaTrack.



Pourquoi ce choix ?
Le monolithe modulaire répond à plusieurs besoins identifiés dans l’arbre d’utilité.

Pour le scénario de **performance (H,H)**, les échanges entre les modules restent principalement internes à l’application et évitent de multiplier les appels réseau entre services.

Pour le scénario de **modifiabilité (M,M)**, la séparation en modules permet d’isoler les intégrations externes du reste de l’application. Le remplacement d’une API de catalogue peut alors être concentré dans le module concerné.

Enfin, cette solution limite la complexité opérationnelle et les coûts d'hébergement par rapport à une architecture composée de nombreux microservices.

Tactiques reliées aux scénarios

Scénario	Tactiques retenues	Rôle dans MangaTrack
Performance (H,H)	Cache, mise à l'échelle horizontale	Redis évite de redemander fréquemment les mêmes données et plusieurs instances de l'API peuvent absorber un pic
Sécurité (H,M)	Authentification, autorisation, chiffrement, journalisation	Un utilisateur accède uniquement à ses propres données et les tentatives interdites sont enregistrées
Modifiabilité (M,M)	Interfaces, ports/adaptateurs, réduction du couplage	Une API externe peut être remplacée sans modifier le reste de MangaTrack
Disponibilité des rappels (H,H)	File persistante, retry, traitement asynchrone	Une panne du fournisseur mail ne fait pas perdre les rappels
Disponibilité du catalogue (H,M)	Cache, timeout, retry, circuit breaker	MangaTrack continue à afficher les données déjà récupérées lorsqu'une API externe tombe en panne
Interopérabilité (M,M)	Interfaces et adaptateurs dédiés	Une nouvelle plateforme externe peut être ajoutée sans toucher aux modules métier

Exemple concret : disponibilité des rappels



Le traitement asynchrone permet de découpler l'utilisation de MangaTrack de l'envoi réel de l'e-mail. Lorsqu'un fournisseur d'e-mails est momentanément indisponible, la tâche reste dans la file et peut être réessayée ultérieurement.

Compromis accepté

Le principal compromis du monolithe modulaire concerne le déploiement. Tous les modules appartiennent à la même application et sont donc généralement déployés ensemble, même lorsqu'un seul module évolue.

MangaTrack accepte cette contrainte en échange d'une architecture plus simple à développer, tester et exploiter. La séparation stricte entre les modules permettra néanmoins d'extraire ultérieurement un service devenu indépendant, par exemple le système de notifications, lorsque la charge ou les besoins le justifieront.

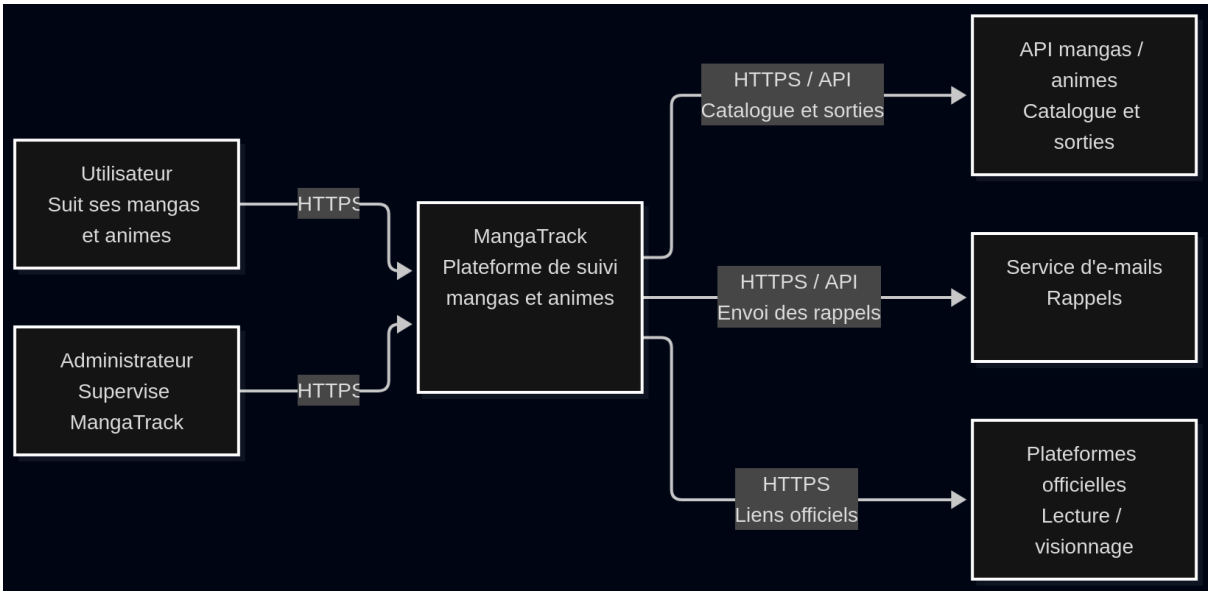
Alternative étudiée : microservices

Une architecture microservices a été envisagée. Elle permettrait notamment de dimensionner séparément le catalogue, les notifications ou les recommandations. Cependant, elle introduirait davantage de communications réseau, de déploiements, de surveillance et de gestion de la cohérence des données. Pour le périmètre actuel de MangaTrack, cette complexité ne répond pas à un besoin prioritaire. Le monolithe modulaire est donc retenu.

Documentation de l'architecture

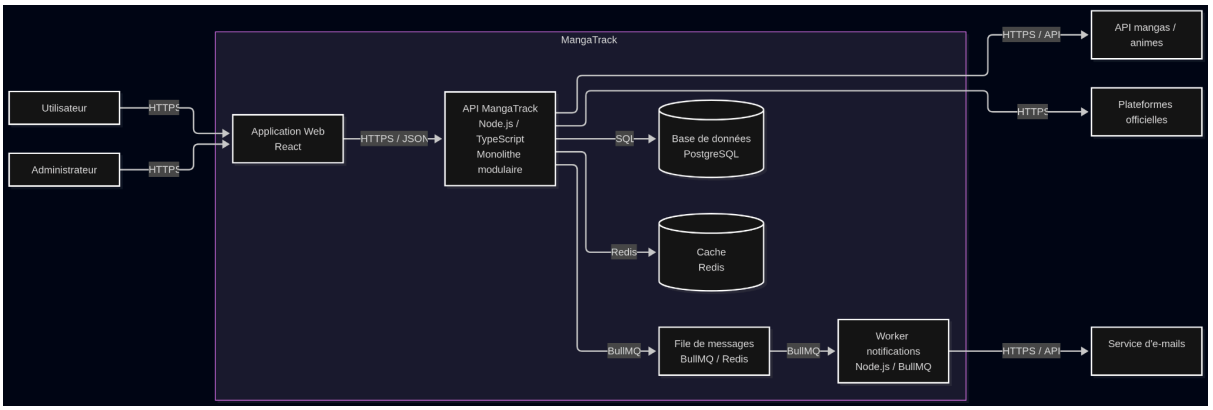
C4 niveau 1 — Contexte

La vue de contexte présente MangaTrack comme un système unique et montre ses utilisateurs ainsi que les systèmes externes avec lesquels il communique.



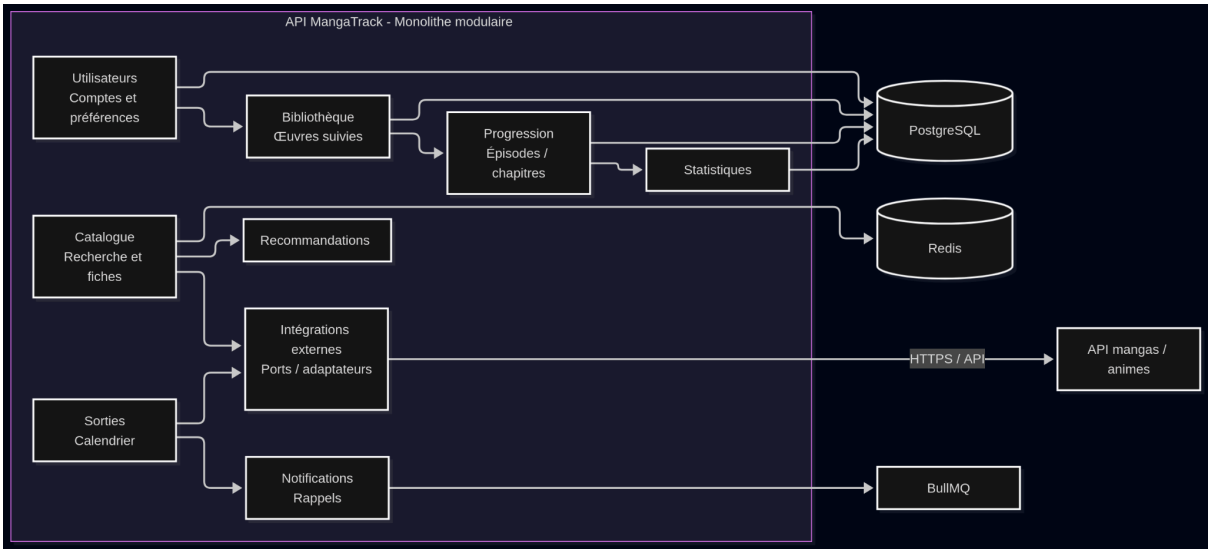
C4 niveau 2 — Conteneurs

La vue de conteneurs montre les principales applications et zones de stockage qui composent MangaTrack.

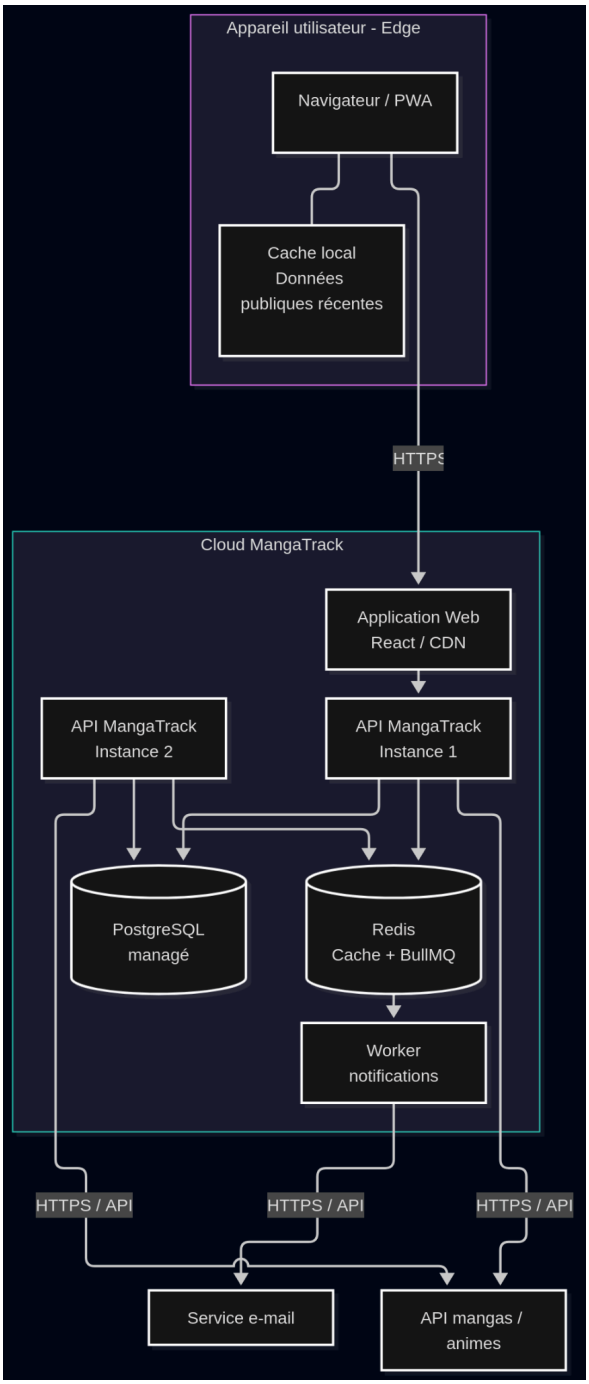


C4 niveau 3 — Composants de l’API MangaTrack

Le niveau 3 détaille l’intérieur du conteneur « API MangaTrack ». Chaque composant correspond à un domaine fonctionnel du monolithe modulaire.



Vue de déploiement cloud et edge



Cloud :

L'API MangaTrack est pensée comme un service sans état. Les informations persistantes restent dans PostgreSQL et Redis. Plusieurs instances de l'API peuvent donc être lancées lorsque la fréquentation augmente, notamment pour répondre au scénario des 5 000 utilisateurs simultanés.

Edge :

L'edge correspond principalement au navigateur ou à la PWA de l'utilisateur. Les fichiers statiques et certaines données publiques récemment consultées peuvent être conservés localement afin de réduire les temps de chargement. Les données personnelles sensibles ne sont pas conservées durablement hors ligne dans le MVP.

Architecture Decision Records

ADR-001 — Monolithe modulaire

Statut : acceptée

Contexte
MangaTrack possède plusieurs domaines fonctionnels mais est développé par une petite équipe avec un budget limité. Les scénarios de performance et de modifiabilité doivent rester satisfaits.

Décision
Utiliser un monolithe modulaire avec des frontières explicites entre utilisateurs, catalogue, bibliothèque, progression, notifications et autres domaines.

Alternative étudiée
Microservices.

Conséquences

- Développement et exploitation plus simples.
- Peu d'appels réseau entre les fonctions.
- Possibilité d'extraire un module plus tard.
  ⇒ Tous les modules sont généralement déployés ensemble.

ADR-002 — Ports et adaptateurs pour les API externes

Contexte
Scénario de modifiabilité (M,M) : une API de catalogue doit pouvoir être remplacée en moins de trois jours.

Décision
Les modules métier ne communiquent jamais directement avec le fournisseur externe. Ils utilisent des interfaces, implémentées par des adaptateurs.

Alternative étudiée
Appeler directement l'API externe depuis les modules Catalogue et Sorties.

Conséquences

- Remplacement plus facile d'un fournisseur.

- Tests possibles sans fournisseur réel.
  ⇒ Davantage d'interfaces et de code intermédiaire.

ADR-003 — Notifications asynchrones

Contexte
Scénario disponibilité (H,H) : une panne du fournisseur mail pendant 30 minutes ne doit entraîner aucune perte de rappel.

Décision
Placer les rappels dans une file BullMQ/Redis et les traiter avec un worker.

Alternative étudiée
Envoyer directement l'e-mail depuis l'API MangaTrack.

Conséquences

- Une panne du fournisseur ne bloque pas MangaTrack.
- Les envois peuvent être retentés.
  ⇒ Certains mails peuvent arriver plus tard.
  ⇒ Il faut empêcher les doublons.

ADR-004 — Cache Redis pour le catalogue

Contexte
Deux scénarios sont concernés : performance (H,H) et indisponibilité du catalogue externe (H,M).

Décision
Conserver temporairement dans Redis les données de catalogue fréquemment demandées.

Alternative étudiée
Interroger systématiquement l'API externe.

Conséquences

- Temps de réponse réduit.
- Moins de requêtes vers l'API externe.
- Certaines fiches restent accessibles lors d'une panne.
  ⇒ Des données peuvent être temporairement anciennes.

ADR-005 — Fonctionnement hors ligne limité

Contexte
Le projet doit traiter la dimension edge et éviter de créer des problèmes de cohérence ou de sécurité.

Décision
Le MVP permet uniquement de conserver localement les fichiers statiques et certaines données publiques déjà consultées. La modification de la progression nécessite une connexion.

Alternative étudiée
Autoriser les modifications de progression hors ligne puis synchroniser au retour du réseau.

Conséquences

- Architecture beaucoup plus simple.
- Aucun conflit de synchronisation à traiter dans le MVP.
- Moins de données personnelles conservées sur l'appareil.
  ⇒ L'utilisateur ne peut pas mettre à jour sa progression hors connexion.

Analyse économique

Estimation simplifiée des coûts d'exploitation

L'analyse reste volontairement indicative. MangaTrack étant un projet étudiant, les coûts représentent des ordres de grandeur relatifs et non un devis réel. L'objectif est de comparer les choix architecturaux.

Poste	Choix retenu	Impact sur le coût
Application Web	Hébergement statique/CDN	Faible
API MangaTrack	Conteneur ou PaaS	Faible à moyen
PostgreSQL	Base managée	Moyen
Redis	Cache et file BullMQ	Faible à moyen
E-mails	Fournisseur externe	Variable selon le volume
API catalogue	Fournisseur externe	Dépend du service utilisé

Bénéfice / Coût

Stratégie	Bénéfice	Coût estimé	Ratio
Cache Redis	80	3 jours	26,7
File asynchrone pour les rappels	70	4 jours	17,5
Ports/adaptateurs pour API externes	60	4 jours	15
Mode hors ligne complet	30	10 jours	3

Le cache présente le meilleur rapport bénéfice/coût car il agit à la fois sur les performances et la dépendance au fournisseur externe. Le mode hors ligne complet apporte moins de valeur pour le MVP et demande beaucoup plus de développement ; il n'est donc pas retenu à ce stade.

Risques restants, dette et évolution

Risque	Conséquence	Réponse prévue
API externe indisponible longtemps	Certaines données deviennent anciennes	Cache, timeout, retry, circuit breaker
Données externes incorrectes	Mauvaise information affichée	Afficher la date de dernière mise à jour
Fournisseur mail indisponible	Retard des rappels	File persistante et retry
Erreur d'autorisation	Exposition de données personnelles	Tests d'accès et journalisation
Croissance du monolithe	Couplage entre modules	Tests d'architecture et frontières strictes
Pic supérieur aux hypothèses	Temps de réponse dégradé	Mise à l'échelle horizontale

Un point de sensibilité important est la durée de conservation du cache. Une durée longue améliore la disponibilité et les performances, mais augmente le risque d'afficher des données anciennes. Il s'agit donc également d'un compromis entre performance et fraîcheur des informations.

Dette architecturale assumée

Plusieurs simplifications sont acceptées pour le MVP. Le système de recommandations repose d'abord sur des genres et tags similaires. Une seule source principale de catalogue peut être utilisée au départ. Le fonctionnement hors ligne reste limité à la consultation de données publiques mises en cache. Enfin, le monolithe impose un déploiement commun à tous les modules.

Ces choix réduisent le coût et le temps de développement mais devront être réévalués lorsque MangaTrack gagnera en utilisateurs ou en fonctionnalités.

Évolution vers une ligne de produits

MangaTrack pourrait évoluer vers plusieurs variantes partageant la même architecture : une version centrée sur les mangas, une version centrée sur les animes ou une déclinaison proposée en marque blanche à des communautés ou acteurs spécialisés.

Les éléments variables pourraient concerner la source de catalogue, le fournisseur de notifications, les règles de recommandation, les fonctionnalités proposées et l'identité graphique. Les ports/adaptateurs et la modularité permettent de localiser ces variations.

Conclusion

L'architecture de MangaTrack repose sur un monolithe modulaire complété par des ports et adaptateurs pour les services externes. Les choix retenus répondent en priorité aux scénarios de performance, disponibilité, sécurité et modifiabilité.

Le cache limite la dépendance aux API externes et améliore les performances, tandis que le traitement asynchrone protège les rappels contre les pannes du fournisseur d'e-mails. L'architecture reste volontairement simple afin de respecter le périmètre du projet et le budget limité.

Certaines fonctions plus complexes, comme la synchronisation complète hors ligne ou les recommandations avancées, sont volontairement reportées. L'architecture retenue permet néanmoins de les intégrer ultérieurement sans remettre en cause l'ensemble du système.

Usage d'outils d'IA générative

Des outils d'IA générative ont été utilisés comme aide à la structuration du dossier, à la reformulation de certains contenus et à la préparation de diagrammes Mermaid. Les choix architecturaux, leur compréhension et leur validation restent sous la responsabilité du groupe.